

Taller de Comparación de Paradigmas de Paralelismo

Sergio Augusto Gelvez Cortes

Michael Alexander Aponte Rodriguez-2222954

Juan David León Delgado 2211587

Daniel Fernando Leal Ayala 2191430

Juan Camilo Jaimes 2221882

7 de abril de 2026

Universidad Industrial de Santander

Colombia

INTRODUCCION A LA COMPUTACION

1 Introducción

Este informe presenta la implementación y análisis de la multiplicación de matrices cuadradas ($C = A \times B$) utilizando el paradigma de programación CUDA con memoria compartida (*Tiled*)[cite: 9, 268]. El objetivo principal es reducir el cuello de botella que representa el acceso a la memoria global mediante la carga colaborativa de datos en la memoria compartida del SM[cite: 309].

2 Entorno de Ejecución

Las pruebas fueron realizadas en un entorno local con las siguientes especificaciones:

- **GPU:** NVIDIA GeForce RTX 4050 Laptop GPU.
- **Arquitectura:** Ada Lovelace (Compute Capability 8.9 / sm_89).
- **Software:** CUDA Toolkit 13.2, Driver 595.97.
- **Compilador:** nvcc V13.2.51.

3 Análisis de Implementación: El Modelo Tiled

A diferencia de la versión *naive*, la implementación *tiled* divide las matrices en bloques de tamaño $TILE \times TILE$ [cite: 273].

1. Cada hilo carga un elemento de la matriz A y uno de la matriz B a arreglos declarados con el calificador `__shared__`[cite: 275, 276, 279].
2. Se utiliza la barrera `__syncthreads()` para asegurar que todos los hilos del bloque hayan terminado la carga antes de proceder al cálculo[cite: 294].
3. Se realiza el producto punto sobre el *tile* y se sincroniza nuevamente antes de cargar el siguiente bloque[cite: 295, 304].

4 Resultados Experimentales

Se evaluó el rendimiento utilizando el protocolo de 5 repeticiones, reportando la mediana de los tiempos[cite: 424].

4.1 Rendimiento vs. Tamaño de TILE

La Tabla 1 muestra el desempeño para el tamaño de tile recomendado ($TILE = 16$) y variaciones.

Cuadro 1: Resultados de ejecución CUDA Tiled (Mediana de 5 repeticiones).

N	TILE	Tiempo (ms)	GFLOPS
512	16	0.32	837.52
1024	16	2.46	873.45
2048	16	19.56	878.25
4096	16	147.03	934.78
4096	8	248.57	552.93
4096	32	143.62	956.95

4.2 Comparación con Versión Naive

Para $N = 1024$, la versión *naive* registró 3.21 ms (669.59 GFLOPS). La versión *tiled* con $TILE = 16$ obtuvo un speedup de **1.30x** sobre la versión *naive* en la misma GPU.

5 Análisis Crítico e Investigación

5.1 Importancia de la Sincronización (__syncthreads)

Se realizaron pruebas eliminando las barreras de sincronización para $N = 1024$:

- **Sin primera barrera:** Resultó en un error máximo absoluto de 23.4, con un 99.95 % de elementos incorrectos debido a condiciones de carrera durante la carga del *tile*.
- **Sin segunda barrera:** Resultó en un 76.44 % de errores, confirmando que hilos rápidos sobrescriben el *tile* antes de que otros terminen su cálculo.

5.2 Análisis de Memoria y Ocupancia (Nsight Compute)

El perfilado para $N = 1024$ reveló:

- **L2 Hit Rate:** Consistentemente superior al 98 % en todas las versiones *tiled*.
- **Ocupancia:** Con $TILE = 32$, la ocupación teórica cae al 66.67 % (frente al 100 % de $TILE = 16$) debido a que el límite de registros y memoria compartida por bloque permite solo 1 bloque activo por SM.

- **Intensidad Aritmética:** La versión $TILE = 32$ alcanzó 8.0 FLOP/byte, lo cual representa el 14.1 % del *ridge point* del hardware (56.8 FLOP/byte), indicando que el algoritmo sigue siendo *memory-bound*.

6 Conclusiones

La implementación *tiled* optimiza significativamente el uso del ancho de banda de memoria. Se observó que aumentar el tamaño del *tile* mejora los GFLOPS hasta un punto donde la reducción de la ocupancia comienza a limitar el desempeño. La correcta sincronización mediante barreras es indispensable para garantizar la integridad de los datos en algoritmos que utilizan memoria compartida.